

Normalizador estadístico vectorizado: comparación de implementación escalar vs AVX2 en NASM x86-64

Yohel Alas Gómez
Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Electrónica
yalas@estudiantec.cr

Bryan Pan
Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Electrónica
bpan@estudiantec.cr

Resumen

Este trabajo compara dos implementaciones en ensamblador NASM x86-64 de un normalizador estadístico que calcula la suma, la media, la varianza poblacional, el mínimo y el máximo de un arreglo de números de punto flotante de precisión simple, y produce su versión normalizada (z-score). La primera versión es escalar y procesa un elemento por instrucción con SSE; la segunda es vectorial y procesa ocho elementos por instrucción con AVX2. Ambas usan el mismo algoritmo y se integran con un programa en C (driver) mediante la convención de llamada System V AMD64. La versión vectorial resuelve explícitamente la alineación de memoria a 32 bytes, la reducción horizontal de los ocho carriles y el remanente cuando N no es múltiplo de 8. Las dos versiones pasan doce casos de prueba, que incluyen todos los casos borde exigidos, y una prueba exhaustiva de 1505 tamaños, con errores relativos menores que 10^{-5} frente a una referencia en doble precisión. Durante la verificación se detectó y corrigió un defecto en arreglos constantes de valores no representables en binario. La versión vectorial ejecuta 7.74 veces menos instrucciones y alcanza un speedup de $7,08\times$ cuando los datos caben en caché L2 ($N = 10^5$). Con 5×10^7 elementos el speedup cae a $3,12\times$, porque el ancho de banda de la memoria principal pasa a ser el factor limitante.

Palabras clave

AVX2, ensamblador x86-64, escalar, SIMD, SISD, vectorización.

I. INTRODUCCIÓN Y OBJETIVOS

Este trabajo implementa dos veces el mismo núcleo de cómputo estadístico en ensamblador NASM para x86-64: una versión *escalar*, que procesa un float por instrucción, y una versión *vectorial* con AVX2, que procesa ocho. Ambas usan el mismo algoritmo, de modo que la diferencia de rendimiento entre ellas se debe al uso de instrucciones SIMD y no a un cambio de método.

I-A. Modelo escalar (SISD) y modelo vectorial (SIMD)

En el modelo SISD cada instrucción opera sobre un único dato. En x86-64, para float, son las instrucciones SSE de sufijo *ss* (*scalar single*): `addss xmm0, xmm1` suma solo el carril 0 (bits 31:0) de los registros. En el modelo SIMD una instrucción opera sobre varios datos empaquetados: con AVX2, los registros `ymm0–ymm15` miden 256 bits y contienen ocho float en ocho *carriles* independientes, y `vaddps ymm0, ymm0, ymm1` hace ocho sumas a la vez Fig 1. Los registros XMM son la mitad baja de los YMM.

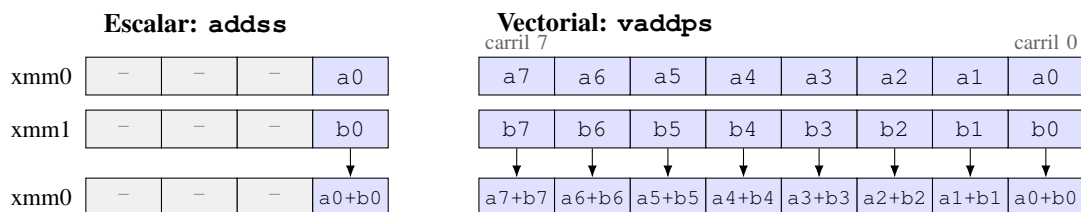


Figura 1. Suma escalar (1 operación útil; en gris, carriles sin usar) frente a suma empaquetada AVX2 (8 operaciones en una instrucción).

El techo teórico de aceleración es entonces $8\times$. No se alcanza porque los $N \bmod 8$ elementos sobrantes deben procesarse con un bucle escalar de cierre, porque los ocho resultados parciales de una suma o un mínimo deben combinarse al final (reducción horizontal), y porque con arreglos grandes el procesador espera datos de la RAM. Los dos primeros puntos son parte de la implementación; el tercero limita el rendimiento con arreglos grandes.

I-B. Planteamiento del problema

Dado un arreglo $x_0, \dots, x_{N-1}$ de float, el programa calcula

$$\mu = \frac{1}{N} \sum_{i=0}^{N-1} x_i, \quad \sigma^2 = \frac{1}{N} \sum_{i=0}^{N-1} (x_i - \mu)^2, \quad y_i = \frac{x_i - \mu}{\sigma}, \quad (1)$$

además de la suma, el mínimo y el máximo. La varianza es poblacional. La entrada es un binario con N (int32) seguido de N float; la salida, el arreglo normalizado y_i y un resumen de texto con los estadísticos. Deben tratarse sin dividir entre cero los casos $N = 0$ y $\sigma = 0$, todos los valores iguales; en ese caso se copia la entrada, según `stats.h`, y deben funcionar $N = 1$, N no múltiplo de 8, valores negativos y extremos. Todo el cómputo es en float, con tolerancia relativa de 10^{-4} .

I-C. Arquitectura de la solución

Un *driver* en C (`src/driver.c`) lee la entrada, reserva los arreglos con `aligned_alloc(32, ...)`, mide el tiempo, calcula $\sigma = \sqrt{\sigma^2}$ con `sqrtof` y escribe los resultados. Los kernels `sum_array`, `compute_stats` y `normalize_array` están en NASM, con las mismas firmas en `stats_scalar.asm` y `stats_vector.asm`, y se enlazan respetando la ABI System V AMD64 [1]. El mismo `driver.o` se enlaza con cada objeto para producir `bin/norm_scalar` y `bin/norm_vector`, de modo que todo lo que no es el kernel es idéntico (figura 2). No se usan intrínsecos de C ni auto-vectorización del compilador. Los diagramas de flujo, de registros y de memoria se entregan en `diagramas.pdf`.

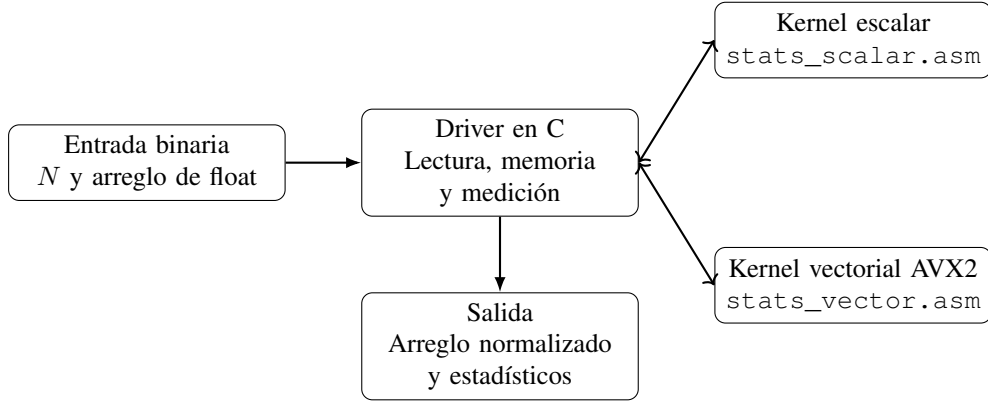


Figura 2. Arquitectura de software y comunicación entre módulos.

I-D. Objetivos

El objetivo general es implementar en NASM una versión escalar y una vectorial AVX2 de un normalizador estadístico, funcionalmente equivalentes, y compararlas en precisión y rendimiento. Los objetivos específicos son:

1. implementar la versión escalar solo con instrucciones escalares SSE;
2. implementar la versión vectorial con AVX2, resolviendo la alineación, la reducción horizontal y el remanente;
3. verificar ambas versiones contra una referencia y entre sí, incluyendo los casos borde, y
4. medir y explicar el *speedup* obtenido.

II. DESCRIPCIÓN DEL ENTORNO DE PRUEBAS

Todas las mediciones a continuación presentadas se realizaron en una sola computadora para mantener los valores comparables entre la metodología escalar y vectorial. El equipo presenta un procesador de la marca Intel, modelo Core i5-1135G7 de la línea de computadores portátiles. Mediante la salida de `lscpu` en terminal se confirmó el soporte de AVX2.

II-A. Hardware implementado

En la tabla II-A se muestra el hardware presente en la computadora utilizada para las mediciones de rendimiento. El programa solo utiliza un hilo, entonces los tamaños de caché que son importantes son por núcleo. Estos datos explican el speedup obtenido en la sección VI.

Tabla I: Resumen de Hardware presente en equipo usado

Elemento	Valor
Procesador	Intel Core i5-1135G7
Núcleos/hilos	4 núcleos, 2 hilos por núcleo (8 CPU lógicas)
Frecuencia	2.40 GHz nominal; de 0.4 a 4.2 GHz con turbo
Caché L1 datos	48 KB por núcleo (192 KiB en 4 instancias)
Caché L1 Instrucciones	32 KB por núcleo (128 KiB en 4 instancias)
Caché L2	1.25 MB por núcleo (5 MiB en 4 instancias)
Caché L3	8 MB, compartida por los 4 núcleos
Contador de Tiempo (TSC)	constant_tsc y nonstop_tsc: avanza a frecuencia fija (2.40 GHz)

II-B. Soporte de extensiones vectoriales

A continuación se mostrará un fragmento de la salida de lscpu. El archivo completo aparece en evidencia/lscpu.txt. Se omitieron algunas etiquetas indicadas con [...]

```
Model name: 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
CPU max MHz: 4200,0000
CPU min MHz: 400,0000
Flags:      fpu vme de pse tsc [...] mmx fxsr sse sse2 [...] constant_tsc [...]
            nonstop_tsc [...] ssse3 [...] fma cx16 [...] sse4_1 sse4_2 [...]
            avx f16c [...] avx2 [...] avx512f avx512dq [...] avx512vl [...]
```

El flag AVX2 confirma la compatibilidad del procesador con las instrucciones utilizadas en el cálculo vectorial.

II-C. Software

En la tabla II se muestran las versiones de las herramientas utilizadas para la ejecución del proyecto.

Tabla II: Versiones de Software de herramientas utilizadas

Herramienta	Versión	Uso
Ubuntu	24.04	sistema operativo
Kernel de Linux	7.0.0-34-generic	
NASM	2.16.01	ensamblado de los kernels: nasm -f elf64 -g -F dwarf
GCC	13.3.0	compilación del driver: gcc -std=gnu11 -Wall -Wextra -O2 -g
GDB	15.1	depuración e inspección de registros YMM (sección VII)
perf	7.0.14	contadores de hardware (sección VI)

Las versiones se puede observar en el archivo en evidencia/versiones.txt

III. IMPLEMENTACIÓN ESCALAR

La versión escalar (asm/scalar/stats_scalar.asm) procesa un elemento por iteración con instrucciones SSE de sufijo ss (movss, addss, subss, mulss, divss, minss, maxss, comiss, ucomiss, cvtsi2ss). Usa a propósito el mismo algoritmo que la vectorial, para que el *speedup* mida solo el efecto de SIMD.

III-A. Convención de llamada

Según la ABI System V AMD64 [1], los enteros y punteros se pasan en `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, y los float en una secuencia aparte, `xmm0`, `xmm1`, ... (tabla III). Por eso en `normalize_array` `mean` y `stddev` llegan en `xmm0` y `xmm1` aunque sean el cuarto y el quinto argumento.

Tabla III: Paso de argumentos. `N` es entero de 32 bits, por eso llega en `esi/edx`.

Función	Argumentos	Retorno
<code>sum_array</code>	<code>rdi = arr, esi = n</code>	<code>xmm0</code>
<code>compute_stats</code>	<code>rdi = arr, esi = n, rdx = mean*, rcx = var*, r8 = min*, r9 = max*</code>	—
<code>normalize_array</code>	<code>rdi = in, rsi = out, edx = n, xmm0 = mean, xmm1 = stddev</code>	—

Los registros *caller-saved* (`rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8`–`r11` y todos los XMM/YMM) pueden modificarse libremente; los *callee-saved* (`rbx`, `rbp`, `r12`–`r15`, `rsp`) deben restaurarse antes de volver. Los kernels solo usan registros *caller-saved*; los `push/pop` de `rbx` y `r12`–`r15` en `compute_stats` vienen del esqueleto y son inofensivos. El índice se lleva en `eax`: escribir un registro de 32 bits pone en cero sus 32 bits altos, así que `rax` sirve directamente en el direccionamiento [`rdi + rax*4`].

III-B. Función `compute_stats`

La asignación de registros en el diagrama 4 de `diagramas.pdf` es: `eax = i`, `xmm0 = suma` → μ , `xmm1 = mínimo`, `xmm2 = máximo`, `xmm3 = temporal`, `xmm4 = (float)N` y `xmm5 = suma de cuadrados`. Si $N \leq 0$ (`test esi, esi + jle`), se escribe 0.0 en los cuatro resultados sin dividir. Si no, el mínimo y el máximo se inicializan con `arr[0]` (seguro porque $N \geq 1$, y evita cargar constantes $\pm\infty$) y se hace la pasada 1:

```
.cs_p1_loop:
    cmp    eax, esi           ; i < n ? (con signo)
    jge    .cs_mean
    movss  xmm3, [rdi + rax*4] ; xmm3 = arr[i] (el puntero avanza 4 bytes)
    addss  xmm0, xmm3         ; suma += arr[i]
    minss  xmm1, xmm3         ; min = min(min, arr[i])
    maxss  xmm2, xmm3         ; max = max(max, arr[i])
    inc    eax
    jmp    .cs_p1_loop
```

Listing 1. Pasada 1 escalar: suma, mínimo y máximo.

- **Una lectura para tres resultados.** Leer el arreglo tres veces triplicaría el tráfico de memoria, que con N grande es el recurso limitante.
- **minss/maxss en vez de comiss + saltos.** Un salto condicional por elemento, dependiente de datos aleatorios, se predice mal con frecuencia; `minss/maxss` no ramifican.
- **No se llama a `sum_array`.** Un call puede destruir `rdx`, `rcx`, `r8` y `r9`, que traen los punteros de salida, y agregaría una pasada extra.

```
.cs_mean:
    cvtsi2ss xmm4, esi        ; xmm4 = (float) n
    ucomiss xmm1, xmm2        ; C4b) min == max ?
    jp     .cs_general        ; algun NaN -> camino general
    je     .cs_const          ; arreglo constante: mean = arr[0], var = 0
.cs_general:
    divss  xmm0, xmm4         ; mean = suma / n
    movss  [rdx], xmm0        ; *mean (y *min, *max en [r8], [r9])
```

Listing 2. Media y detección de arreglo constante.

El bloque C4b se agregó tras una falla en las pruebas: con 1001 copias de 0.1f, la suma acumula redondeo, la media no es exactamente x_0 y la varianza sale $\approx 10^{-13}$ en vez de 0, de modo que la normalización no detecta $\sigma = 0$. La comparación `mín = máx` es exacta (ambos son elementos del arreglo) y es verdadera si y solo si todos los valores son iguales; en ese caso se escriben $\mu = x_0$ y $\sigma^2 = 0$. El `jp` evita confundir un NaN con igualdad, porque `ucomiss` indica “no ordenado” con `ZF = PF = 1`.

```

.cs_p2_loop:
    cmp     eax, esi
    jge     .cs_var
    movss   xmm3, [rdi + rax*4]    ; arr[i]
    subss   xmm3, xmm0            ; arr[i] - mean (destino destructivo)
    mulss   xmm3, xmm3            ; (arr[i] - mean)^2
    addss   xmm5, xmm3
    inc     eax
    jmp     .cs_p2_loop
.cs_var:
    divss   xmm5, xmm4            ; var = suma / n
    movss   [rcx], xmm5

```

Listing 3. Pasada 2 escalar y varianza.

La varianza usa dos pasadas y no la fórmula $\sigma^2 = \overline{x^2} - \mu^2$, que ahorra una lectura pero resta dos números de magnitud $\mu^2 + \sigma^2$, cada uno con un error de al menos $u(\mu^2 + \sigma^2)$, con $u = 2^{-24} \approx 5,96 \times 10^{-8}$ [2]. El error relativo de σ^2 sería entonces del orden de $u(\mu^2 + \sigma^2)/\sigma^2$: con datos $1000 + U(0, 1)$ ($\sigma^2 = 1/12$), cerca de 0,7 (cancelación catastrófica). Restando la media primero, los sumandos son pequeños y positivos.

El formato SSE tiene dos operandos y el destino es también la primera fuente: `subss xmm3, xmm0` sobrescribe `xmm3`. Por eso cada elemento se carga en el temporal `xmm3` y la media se usa siempre como segundo operando, de modo que `xmm0` no cambia en toda la pasada.

III-C. *normalize_array*

```

xorps     xmm2, xmm2
comiss    xmm1, xmm2            ; stddev == 0 (o NaN) ?
je        .norm_copy_loop      ; si: out[i] = in[i]
.norm_loop:
    cmp     eax, edx
    jge     .norm_done
    movss   xmm2, [rdi + rax*4]  ; in[i]
    subss   xmm2, xmm0           ; in[i] - mean
    divss   xmm2, xmm1           ; (in[i] - mean) / stddev
    movss   [rsi + rax*4], xmm2  ; out[i]
    inc     eax
    jmp     .norm_loop

```

Listing 4. Bucle principal de `normalize_array` escalar.

Si $\sigma = 0$, `comiss` pone `ZF = 1` y el código salta a una rama que copia la entrada (`movss` de `[rdi+rax*4]` a `[rsi+rax*4]`), así que nunca se divide entre cero; un σ NaN también cae ahí. Como `mean` y `stddev` nunca son destino, se quedan en `xmm0/xmm1` todo el bucle sin copiarlos a la pila.

Cada bucle escalar tiene entre 6 y 8 instrucciones, de las cuales 4 son de control (`cmp`, `jge`, `inc`, `jmp`); en total son 30 instrucciones por elemento sumando los cuatro recorridos. Con `objdump -d` se verificó que el objeto no contiene instrucciones empaquetadas (salvo el modismo `xorps reg, reg` para poner en cero) ni VEX; por eso no necesita `vzeroupper`.

IV. IMPLEMENTACIÓN VECTORIAL (AVX2)

La versión vectorial (`asm/vector/stats_vector.asm`) conserva la ABI, los casos borde y el algoritmo de la escalar, pero procesa ocho float por instrucción en registros YMM. A cambio debe resolver tres problemas que la escalar no tiene: la alineación de memoria, la combinación de los ocho resultados parciales y los elementos sobrantes.

IV-A. Elección de AVX2

SSE (128 bits, 4 float) limitaría el techo a $4\times$ y desaprovecharía la mitad del ancho disponible. AVX-512 (512 bits, 16 float) está en el procesador de pruebas, pero no en muchos procesadores actuales (Intel lo deshabilitó en los núcleos de escritorio desde la 12.^a generación), y con arreglos grandes el límite es la memoria, no el cómputo.

AVX2 (256 bits, 8 float) está disponible en prácticamente todo x86-64 desde 2013, es lo que pide el enunciado y aporta la forma `vbroadcastss ymm, xmm` con fuente en registro (en AVX1 la fuente solo podía ser memoria). No se usó FMA (`vfmadd231ps`): es una extensión aparte y redondea distinto que la secuencia multiplicación + suma de la versión escalar.

Las instrucciones AVX usan codificación VEX (prefijo `v`), con tres operandos y destino no destructivo (`vsubps ymm7, ymm7, ymm5`), y las VEX de 128 bits ponen en cero los bits 255:128 de su YMM destino, de modo que tras reducciones y remanentes no quedan mitades altas con basura.

IV-B. Patrón de tres fases

Con $N = 8q + r$, $0 \leq r < 8$, cada recorrido del arreglo tiene un bucle vectorial de q iteraciones ($i = 0, 8, \dots$), una reducción horizontal y un bucle de remanente de r iteraciones escalares. El límite $8q$ se obtiene con `and r10d, ~7`, que borra los tres bits bajos de N (que valen $N \bmod 8$): para $N = 1001 = 1111101001_2$ resulta $1111101000_2 = 1000$, es decir, 125 iteraciones vectoriales y 1 de remanente. Con $N < 8$ el bucle vectorial no itera y solo se ejecuta el remanente.

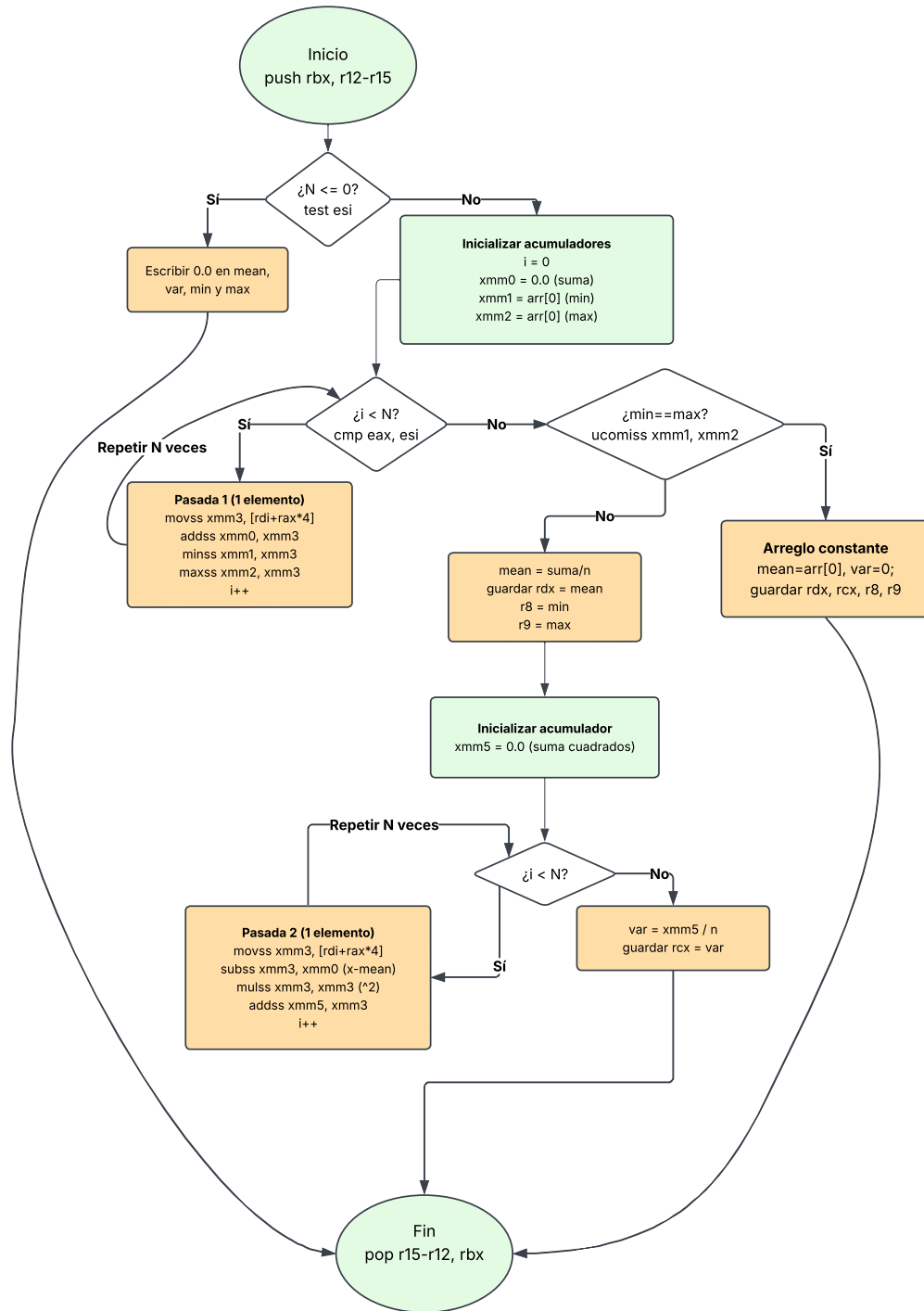


Figura 3. Flujo de control de `compute_stats` escalar: un bucle por pasada.

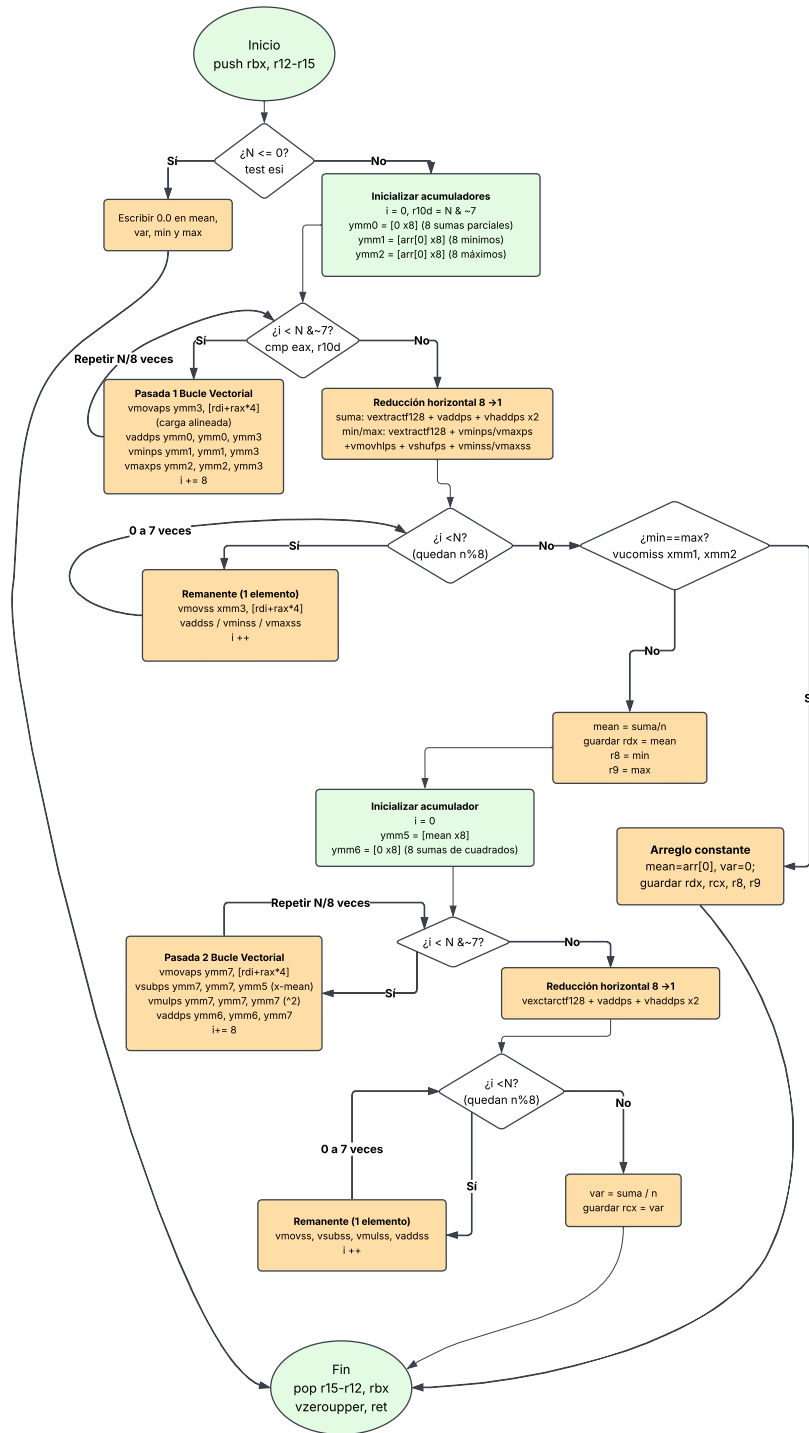


Figura 4. Flujo de control de `compute_stats` vectorial: un bucle de 8 elementos, una reducción y un remanente de 0 a 7 iteraciones por pasada.

IV-C. `compute_stats` vectorial

La inicialización crea ocho acumuladores por resultado: `vxorps ymm0, ymm0, ymm0` (8 sumas en 0) y `vbroadcastss ymm1, [rdi], vbroadcastss ymm2, [rdi]` (8 mínimos y 8 máximos iguales a `arr[0]`). El límite se guarda en `r10d` y no en `ecx` porque en esta función `rcx` trae el puntero `var*`.

```
.cs_pl_loop:
    cmp     eax, r10d          ; i < n & ~7 ?
```

```

jge      .cs_pl_reduce
vmovaps  ymm3, [rdi + rax*4]    ; arr[i..i+7] (carga ALINEADA de 32 bytes)
vaddps   ymm0, ymm0, ymm3      ; carril k: suma[k] += arr[i+k]
vminps   ymm1, ymm1, ymm3      ; carril k: min[k] = min(min[k], arr[i+k])
vmaxps   ymm2, ymm2, ymm3
add      eax, 8                 ; el puntero avanza 32 bytes
jmp      .cs_pl_loop

```

Listing 5. Pasada 1 vectorial: una carga y tres operaciones sobre 8 elementos.

El cuerpo es el del listado 1 con cada *ss* reemplazada por *ps* y el incremento cambiado a 8: las mismas 8 instrucciones cubren ahora ocho elementos. Al terminar, el carril k de *ymm0* tiene la suma de los elementos $k, k + 8, k + 16, \dots$ (tabla IV).

IV-C1. Reducción horizontal: Las instrucciones empaquetadas son *verticales*, combinan el carril k de dos registros; para obtener un solo resultado hay que combinar carriles del mismo registro:

```

vextractf128 xmm3, ymm0, 1      ; xmm3 = carriles 4-7 (mitad alta)
vaddps   xmm0, xmm0, xmm3      ; {s0+s4, s1+s5, s2+s6, s3+s7}
vhaddps  xmm0, xmm0, xmm0      ; sumas de pares adyacentes
vhaddps  xmm0, xmm0, xmm0      ; xmm0[0] = suma total

vextractf128 xmm3, ymm1, 1      ; minimo: mitad alta
vminps   xmm1, xmm1, xmm3      ; 4 candidatos
vmovhlps xmm3, xmm3, xmm1      ; xmm3[0..1] = xmm1[2..3]
vminps   xmm1, xmm1, xmm3      ; 2 candidatos
vshufps  xmm3, xmm1, xmm1, 0x55 ; 0x55 = 01010101b: todos = xmm1[1]
vminss   xmm1, xmm1, xmm3      ; xmm1[0] = minimo (maximo: identico)

```

Listing 6. Reducciones de la suma ($8 \rightarrow 4 \rightarrow 1$) y del mínimo ($8 \rightarrow 4 \rightarrow 2 \rightarrow 1$).

Tabla IV: Carriles de *ymm0/xmm0* con $x_i = i + 1$, $N = 16$ (mismos valores que en la sesión de GDB). El total es $\sum_{i=1}^{16} i = 136$.

Momento	0	1	2	3	4	5	6	7
Tras la iteración 1	1	2	3	4	5	6	7	8
Tras la iteración 2	10	12	14	16	18	20	22	24
vextractf128 + vaddps	28	32	36	40				
1.º vhaddps	60	76	60	76				
2.º vhaddps	136	136	136	136				

vextractf128 va primero porque casi todas las instrucciones AVX de 256 bits operan por separado en cada mitad de 128 bits: *vhaddps ymm* nunca cruzaría de una mitad a la otra. Para la suma, dos *vhaddps* son la forma más corta. Una alternativa es repetir el esquema del mínimo (*vmovhlps* + *vaddps* + *vshufps* + *vaddss*), algo más rápida porque *vhaddps* se descompone internamente en dos permutaciones y una suma; pero la reducción se ejecuta una vez por pasada (4 instrucciones frente a 100 000 del bucle con $N = 10^5$), así que la diferencia no es medible y se prefirió la versión más clara. Para mínimo y máximo no existe instrucción horizontal, por lo que se parte el vector a la mitad en cada paso.

IV-C2. Remanente:

```

.cs_pl_tail:
cmp      eax, esi              ; i vale n & ~7 al entrar
jge      .cs_mean
vmovss   xmm3, [rdi + rax*4]   ; un solo float
vaddss   xmm0, xmm0, xmm3
vminss   xmm1, xmm1, xmm3
vmaxss   xmm2, xmm2, xmm3
inc      eax
jmp      .cs_pl_tail

```

Listing 7. Remanente de la pasada 1 (N mód 8 elementos).

El remanente va *después* de la reducción porque las instrucciones *ss* trabajan en el carril 0, que es donde la reducción dejó los resultados. No se lee un último bloque incompleto de 8 aunque el driver rellena con ceros hasta

un múltiplo de 32 bytes: esos ceros alterarían el mínimo (con datos positivos daría 0) y la suma de cuadrados, porque $(0 - \mu)^2 \neq 0$, y en `normalize_array` se escribiría fuera de los N elementos. Evitarlo de forma vectorial requiere cargas enmascaradas (`vmaskmovps`, o los registros k de AVX-512); el bucle escalar es más simple y cuesta como máximo 7 iteraciones.

IV-C3. Media y pasada 2: La media y la detección de arreglo constante son las de la versión escalar (listado 2) con `vcvtss2ss`, `vucomiss` y `vdivss`. Para la pasada 2, `vbroadcastss ymm5, xmm0` copia μ a los ocho carriles y el bucle acumula $(x - \mu)^2$ en `ymm6` con `vsubps`, `vmulps` y `vaddps`; siguen la misma reducción de la suma y un remanente con `vsubss`, `vmulss` y `vaddss` que usa μ directamente del carril 0 de `xmm0`.

Los ocho acumuladores también mejoran la precisión. En una suma secuencial de N términos, $|\hat{S} - S| \leq (N - 1)u \sum |x_i| + O(u^2)$ [2]; con ocho carriles cada uno suma $N/8$ términos, y la cota baja a cerca de $(N/8 + 10)u \sum |x_i|$ (más los 3 niveles de la reducción y hasta 7 sumas del remanente), unas ocho veces menor. Por eso, con $N = 5 \times 10^7$, la varianza escalar se desvía un 12.6 % de la referencia y la vectorial un 0.55 %.

IV-D. `normalize_array` vectorial

```

vbroadcastss ymm2, xmm0      ; ymm2 = [mean  x8]
vbroadcastss ymm3, xmm1      ; ymm3 = [stddev x8]
.norm_vec_loop:
    cmp     eax, ecx          ; ecx = n & ~7
    jge     .norm_scalar_tail
    vmovaps ymm4, [rdi + rax*4] ; in[i..i+7]
    vsubps  ymm4, ymm4, ymm2    ; 8 restas
    vdivps  ymm4, ymm4, ymm3    ; 8 divisiones
    vmovaps [rsi + rax*4], ymm4 ; out[i..i+7] (guardado alineado)
    add     eax, 8
    jmp     .norm_vec_loop

```

Listing 8. Bucle vectorial de `normalize_array`.

Aquí sí se usa `ecx` como límite, porque la función no recibe nada en `rcx`. Los temporales son `ymm2–ymm4`, de modo que `xmm0` y `xmm1` siguen intactos y el remanente (`vsubss/vdivss`) los usa sin guardarlos en la pila. El caso $\sigma = 0$ se detecta con `vucomiss` y lleva a una rama de copia, también vectorial con remanente. Se usa `vdivps` y no la multiplicación por $1/\sigma$, más rápida, porque $x \cdot \text{fl}(1/\sigma)$ redondea dos veces y no coincidiría bit a bit con el `divss` de la versión escalar.

IV-E. Alineación y `vzeroupper`

`vmovaps` de 256 bits exige una dirección múltiplo de 32; si no lo es, genera una excepción `#GP (segmentation fault)`. El camino principal la usa, como pide el enunciado, y es seguro por construcción: el driver reserva ambos arreglos con `aligned_alloc(32, ...)` (en GDB, las bases `0x55555555a4c0` y `0x555555559480` tienen sus cinco bits bajos en 0), y el bucle avanza la dirección de 32 en 32 bytes, de modo que todas siguen siendo múltiplos de 32. El remanente usa `vmovss`, que solo requiere la alineación natural de un float. El diagrama 5 de `diagramas.pdf` muestra este avance: 4 bytes por iteración en la escalar y 32 en la vectorial. El `sum_array` del profesor usa `vmovups` y se dejó sin cambios; con datos alineados su costo es el mismo, pero no detecta una dirección mal alineada.

Cada función vectorial termina con `vzeroupper`, que limpia los bits 255:128 de todos los YMM. Así, si el llamador ejecuta instrucciones SSE sin VEX, no paga la penalización por transición AVX→SSE.

IV-F. Comparación de ambas versiones

Tabla V: Diferencias estructurales entre las dos versiones.

Aspecto	Escalar	Vectorial
Elementos por iteración / avance	1 / 4 bytes	8 / 32 bytes
Instrucciones por elemento (4 bucles)	30	$30/8 \approx 3,75$
Acumuladores de suma	1	8 (uno por carril)
Reducción horizontal	no	vextractf128, vhaddps, vmovhlps, vshufps
Remanente	no	$N \bmod 8$ iteraciones escalares
Difusión de parámetros	no	vbroadcastss
Formato	SSE, 2 operandos	VEX, 3 operandos
Acceso a memoria	movss	vmovaps alineada
vzeroupper	no	sí

Como cada bucle vectorial tiene las mismas instrucciones que su par escalar y procesa ocho elementos, el número de instrucciones por elemento cae casi exactamente a la octava parte.

V. CASOS DE PRUEBA

Ambas versiones pasan los doce casos de precisión, que cubre todos los casos borde del enunciado, con errores entre 10^{-10} y 10^{-5} frente a una tolerancia de 10^{-4} . También pasan una prueba exhaustiva de 1505 casos por versión sin ninguna escritura fuera del arreglo de salida.

V-A. Metodología

La referencia es `verify_reference.py`, que calcula los estadísticos en doble precisión y los compara con el `.stats.txt` del driver. Como no revisa el arreglo normalizado ni compara las versiones entre sí, se escribió `tools/correctud.py`, que para cada caso genera la entrada con semilla fija, ejecuta ambos binarios y compara los estadísticos y el arreglo contra la referencia y entre versiones. Para los estadísticos se usa el error relativo de `verify_reference.py`, $e_{\text{rel}} = |a - b|/|b|$ (absoluto si $|b| < 10^{-12}$); para el arreglo,

$$e_{\text{norm}} = \max_i \frac{|\hat{y}_i - y_i|}{\max(1, |y_i|)}, \quad (2)$$

porque muchos *z-scores* son cercanos a 0 y en ellos el error relativo puro crece sin sentido. Un caso pasa si todas las métricas son $\leq 10^{-4}$ en ambas versiones y entre ellas. Esa tolerancia, fijada por el enunciado, es unas 1700 veces la unidad de redondeo de `float` ($u = 2^{-24} \approx 5,96 \times 10^{-8}$) y absorbe las diferencias por el distinto orden de las sumas.

Tabla VI: Casos de prueba. Cubren los casos borde del enunciado: $N = 0$, $N = 1$, N no múltiplo de 8, valores iguales, negativos y extremos.

Caso	N	Datos	Qué verifica
N0	0	vacío	error controlado, sin división entre cero
N1	1	aleatorio $[-100, 100]$	$\sigma = 0$ y rama de copia; solo remanente
N7	7	aleatorio	0 iteraciones vectoriales, solo remanente
N8, N16	8, 16	aleatorio	1 y 2 bloques exactos, sin remanente
N15	15	aleatorio	1 bloque + remanente máximo (7)
N1000	1000	aleatorio	tamaño de verificación del enunciado
const5	1000	todos 5,0	$\sigma = 0$ con valor exacto en binario
const01	1001	todos 0,1	$\sigma = 0$ con valor no exacto en binario
neg	1001	$[-1000, -1]$	negativos; máximo negativo; remanente de 1
ext	1001	$\pm[10^{15}, 10^{16}]$	valores extremos
edge	1000	$\pm 10^6, 0, \pm 10^{-4}, \pm 1$	magnitudes muy distintas mezcladas

V-B. Resultados

Tabla VII: Media y varianza de referencia y obtenidas (9 cifras significativas). El mínimo y el máximo coinciden exactamente con la referencia en todos los casos, porque `minss/vminps` seleccionan un elemento sin redondear.

Caso	μ			σ^2		
	Referencia	Escalar	Vectorial	Referencia	Escalar	Vectorial
N0	0	0	0	0	0	0
N1	16.2304268	16.2304268	16.2304268	0	0	0
N7	-3,22382549	-3,22382569	-3,22382569	2771.30314	2771.30298	2771.30298
N8	-25,5031893	-25,5031891	-25,5031891	3316.83171	3316.83203	3316.83203
N15	6.82065466	6.82065630	6.82065535	4072.35773	4072.35718	4072.35767
N16	8.78149293	8.78149223	8.78149223	1935.23747	1935.23743	1935.23755
N1000	-0,280658448	-0,280658931	-0,280659169	3316.97228	3316.97217	3316.97241
const5	5	5	5	0	0	0
const01	0.100000001	0.100000001	0.100000001	0	0	0
neg	-482,727528	-482,727417	-482,727539	86847.3885	86847.4688	86847.3828
ext	1,18832375e14	1,18832316e14	1,18832409e14	3,60333234e31	3,60333155e31	3,60333228e31
edge	-0,001	-0,00100000005	-0,00100000005	2,86000000e11	2,86000677e11	2,85999923e11

Tabla VIII: Errores máximos: e_{rel} sobre los seis estadísticos y e_{norm} (2) del arreglo. Tolerancia: 10^{-4} .

Caso	Escalar		Vectorial		Esc. vs. vec.	Resultado
	estadísticos	arreglo	estadísticos	arreglo		
N0	0	0	0	0	0	PASA
N1	7,2e-10	0	7,2e-10	0	0	PASA
N7	7,4e-8	1,2e-7	7,4e-8	1,2e-7	0	PASA
N8	9,5e-8	7,2e-8	9,5e-8	7,2e-8	0	PASA
N15	2,4e-7	1,5e-7	1,0e-7	8,3e-8	1,1e-7	PASA
N16	8,0e-8	7,2e-8	8,0e-8	1,5e-7	1,1e-7	PASA
N1000	1,7e-6	1,2e-7	2,6e-6	1,2e-7	1,5e-8	PASA
const5	0	0	0	0	0	PASA
const01	9,6e-6	0	8,8e-7	0	0	PASA
neg	9,2e-7	9,1e-7	6,5e-8	2,0e-7	9,9e-7	PASA
ext	5,1e-7	2,1e-7	2,9e-7	1,1e-7	2,4e-7	PASA
edge	2,4e-6	1,2e-6	2,7e-7	9,0e-8	1,3e-6	PASA

V-B0a. Casos borde.: Con $N = 0$ ambos kernels toman la salida temprana y escriben 0.0; el driver imprime un aviso por `stderr` y termina normalmente. Con $N = 1$, $\sigma = 0$ y la normalización copia la entrada en lugar de calcular $0/0$; el error de $7,2 \times 10^{-10}$ proviene de imprimir el `float` con 9 cifras. Con `neg`, el máximo es $-1,1785$; un máximo inicializado en 0 en lugar de `arr[0]` habría fallado. Con `ext` la suma de cuadrados llega a $\approx 3,6 \times 10^{34}$, todavía bajo el máximo de `float` ($3,4 \times 10^{38}$), y el error relativo no crece con la escala.

V-B0b. Remanente.: Con $N = 7$ la versión vectorial ejecuta solo el remanente, con las mismas operaciones y en el mismo orden que la escalar, y los resultados son idénticos bit a bit. Con $N = 15$ (remanente máximo) el orden de las sumas ya difiere y las versiones discrepan en el último dígito ($\sigma^2 = 4072,35718$ frente a $4072,35767$), con diferencia de $1,1 \times 10^{-7} \approx 2u$.

V-B0c. Error relativo cerca de cero.: El mayor error de N1000 ($2,6 \times 10^{-6}$) está en la suma y la media: los datos en $[-100, 100]$ suman casi 0 ($\mu \approx -0,28$) y el error relativo de un resultado pequeño se amplifica. Medido contra $\sum |x_i| \approx 5 \times 10^4$, el error es del orden de 10^{-8} . En `edge`, con magnitudes que difieren en diez órdenes, la varianza vectorial es casi diez veces más exacta que la escalar por el efecto de los ocho acumuladores (sección IV-C).

V-C. Hallazgo: arreglo constante con valores no representables

La primera versión del código pasaba `const5` pero fallaba `const01` (tabla IX). 0.1 no es representable en binario; al sumar 1001 copias se acumula redondeo y la media queda ligeramente por encima o por debajo de x_0 , según el orden de las sumas. Entonces $x_i - \mu$ es pequeño pero no nulo, $\sigma > 0$, no se toma la rama de copia

y $y_i = (x_0 - \mu)/|x_0 - \mu| = \pm 1$, con signo opuesto en cada versión. Con 5,0 no ocurría porque $1000 \times 5,0$ es exacto. La corrección es el bloque C4b/A6b (listado 2): si mín = máx, se escriben $\mu = x_0$ y $\sigma^2 = 0$. Cuesta tres instrucciones por llamada, fuera de los bucles, y `objdump` confirma que el resto del código no cambió.

Tabla IX: Caso `const01` antes y después de la corrección.

Versión	μ	σ^2	σ	y_i
Escalar, sin C4b	0.0999990478	$9,1 \times 10^{-13}$	$9,5 \times 10^{-7}$	+1
Vectorial, sin A6b	0.1000000909	$8,0 \times 10^{-15}$	$8,9 \times 10^{-8}$	-1
Ambas, corregidas	0.100000001	0	0	0.100000001 (copia)

VI. RESULTADOS DE RENDIMIENTO

Los resultados obtenidos mostraron que la versión vectorial es entre 3.1 y 7.1 veces más rápida que la escalar. El mejor speedup se obtuvo con $N = 1 \times 10^5$, cuando los datos caben en la caché L2. Con $N = 5 \times 10^7$ el speedup cae a $3,12\times$, esto debido al ancho de banda de la memoria.

VI-A. Metodología

- Solo se mide el núcleo de computo (`sum_array`, `compute_stats`, `sqrtrf` y `normalize_array`), sin la lectura ni la escritura de archivos. El driver toma el tiempo con `clock_gettime(CLOCK_MONOTONIC)` y los ciclos con `rdtsc`, justo antes y después de las llamadas.
- Repeticiones: 30 por tamaño y versión, dentro de una misma ejecución del programa. Se reporta el promedio y la desviación estándar muestral (divisor $k - 1$).
- Cambio al driver: El driver original solo acumulaba el tiempo total y reportaba el promedio, sin forma de calcular la desviación. Se modificó para guardar el tiempo y los ciclos de cada repetición y escribirlos en un archivo aparte (`.timing.txt`), sin alterar el archivo `.stats.txt` que lee `verify_reference.py`.
- Datos: Entradas aleatorias en $[-100, 100]$ generadas con `tools/gen_input.py`. El script `tools/bench.py` ejecuta las 8 combinaciones (4 tamaños $\times$ 2 versiones) y genera la tabla y el gráfico.
- Speedup = tiempo escalar / tiempo vectorial, con los promedios de cada tamaño.

VI-B. Tiempos de ejecución y Speedup

En la tabla X se muestra el tamaño de cada archivo de prueba generado en base al tamaño de N usado, frente a la capacidad de cada caché del sistema, esto es importante para analizar el porqué en cierto tamaño el rendimiento empieza a empeorar.

Tabla X: Tamaño de archivos de N datos comparados con el tamaño memoria disponible

N	Memoria utilizada	Dónde caben los datos
10^3	8 KB	L1 (48 KB)
10^5	800 KB	L2 (1.25 MB)
10^6	8 MB	límite de la L3 (8 MB)
5×10^7	400 MB	solo RAM

En la tabla XI se muestra la latencia en ms y ciclos de cada una de las metodologías.

Tabla XI: Comparación de latencia y speedup en metodologías Escalar y Vectorial

N	Escalar (ms)	Vectorial (ms)	Ciclos Escalar	Ciclos Vectorial	Speedup
10^3	0.003775 ± 0.000031	0.000588 ± 0.000018	9 062	1 357	6.42×
10^5	0.3996 ± 0.0049	0.0565 ± 0.0013	966 407	136 530	7.08×
10^6	3.735 ± 0.183	0.673 ± 0.100	9 033 897	1 626 263	5.55×
5×10^7	184.66 ± 2.25	59.27 ± 1.22	446 659 750	143 353 188	3.12×

Para obtener estos datos, se realizaron 30 repeticiones por cada valor de N. La latencia en ciclo se mide en base a la frecuencia nominal de 2.40 GHz y no a la frecuencia turbo del sistema. En la figura 5 se aprecia como a partir de una cantidad de elementos de entrada el speed up cae.

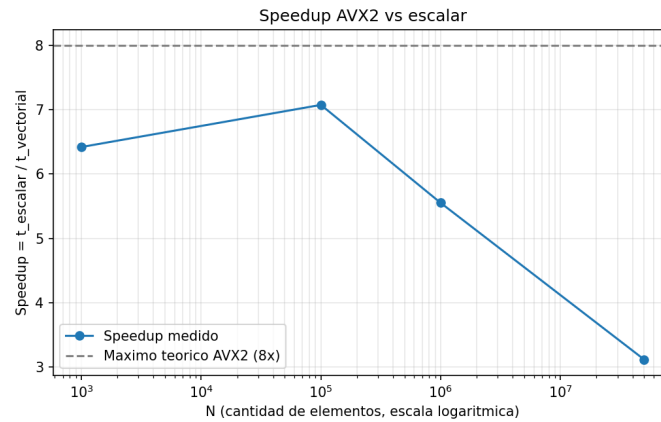


Figura 5. Speedup en base a la cantidad de elementos de entrada

La curva tiene tres zonas, que coinciden con la jerarquía de caché de la sección II:

- $N = 10^3$ (6.42x), costos fijos. El vectorial dura unos 1 360 ciclos, y una parte de ellos no se acelera con SIMD: las tres llamadas, las reducciones horizontales, los vbroadcastss, las divisiones escalares y el sqrtf del driver. Con un trabajo tan corto esa parte pesa más.
- $N = 10^5$ (7.08x), límite por cómputo. Los 800 KB caben en la L2 de un núcleo, los costos fijos se diluyen y la memoria no frena: es el mejor caso.
- $N = 10^6$ y 5×10^7 (5.55x y 3.12x), límite por memoria. Con 8 MB los datos alcanzan el tamaño de la L3 compartida y parte del tráfico ya va a la RAM; por eso esta medición tiene la mayor dispersión ($\pm 15\%$ en la vectorial). Con 400 MB todo viene de la RAM.

En cada llamada el programa lee el arreglo de entrada 4 veces (sum-array, las dos pasadas de compute-stats y normalize-array) y escribe una vez la salida: al menos $5 \times 200 \text{ MB} = 1.0 \text{ GB}$ de tráfico con $N = 5 \times 10^7$. La versión vectorial lo mueve en 59.27 ms (16.9 GB/s) y la escalar en 184.66 ms (5.4 GB/s). Los 16.9 GB/s están cerca del máximo práctico de un solo núcleo; el máximo teórico de la memoria solo se aproxima usando varios núcleos a la vez.

VI-C. Contadores de hardware (perf stat)

Perf mide el programa completo, incluida la lectura del archivo, para que las mediciones sean significativas se corrió el programa 3000 veces para $N = 10^5$ y 30 veces para $N = 5 \times 10^7$ para que el tiempo que domine sea el de la ejecución de los programas. Los resultados se muestran en la tabla XII

Tabla XII: Resultados de cantidad de ciclos, instrucciones y misses de caché durante ejecución

Versión	N	Reps	Ciclos	Instrucciones	IPC	Misses totales	Misses por repetición
Escalar	10^5	3000	4 514 438 197	9 012 915 508	2.00	85 366	28
Vectorial	10^5	3000	665 010 266	1 164 035 559	1.75	183 562	61
Escalar	5×10^7	30	23 909 916 240	45 798 373 646	1.92	219 869 475	7.3 millones
Vectorial	5×10^7	30	7 889 290 296	6 374 922 175	0.81	446 934 382	14.9 millones

Se puede ver como la vectorización logra reducir las instrucciones casi 8 veces. Con $N = 10^5$ la versión vectorial ejecutó 7.74x menos instrucciones. Con datos en la caché, el IPC explica porqué 7x, de manera vectorial se completan algo menos de instrucciones por ciclo, 1.75 frente a 2, esto se debe a que vdivps sobre 8 floats es más costosa que divss. También, se aprecia que al aumentar la cantidad de datos, ya no entran en la caché. Esto se aprecia al comparar el IPC de $N = 10^5$ y $N = 5 \times 10^7$. Con la versión escalar apenas hay diferencia de IPC, sigue limitada por el poder de cómputo. En cambio, entre versiones de la vectorial el IPC cae significativamente, esto ocurre por la latencia de la memoria principal. Por otro, se ve como aumentan los fallos en caché, también, por lo antes mencionado.

VII. EVIDENCIA DE LA SESION DE GDB

VII-A. Vectorial

Se muestra a continuación como tras cada vaddps los 8 carriles de ymm0 se suman en paralelo y después del bucle de normalización en la salida se obtienen los valores normalizados correctos.

```
+break sum_array.sum_vec_loop
Punto de interrupcion 1 at 0x1d1f: file asm/vector/stats_vector.asm, line 58.
+break compute_stats.cs_p1_reduce
Punto de interrupcion 2 at 0x1d9b: file asm/vector/stats_vector.asm, line 151.
+break normalize_array.norm_done
Punto de interrupcion 3 at 0x1f25: file asm/vector/stats_vector.asm, line 342.
+run
Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:58
58      cmp      eax, ecx
+info registers rdi rsi rax rcx
rdi      0x55555555a4c0      93824992257216
rsi      0x10              16
rax      0x0               0
rcx      0x10              16
+stepi 4
62      add      eax, 8      ; avanza 8 floats = 32 bytes
+x/i $pc
=> 0x55555555d2c <sum_array.sum_vec_loop+13>: add $0x8,%eax
+p $ymm1.v8_float
$1 = {1, 2, 3, 4, 5, 6, 7, 8}
+p $ymm0.v8_float
$2 = {1, 2, 3, 4, 5, 6, 7, 8}
+continue
Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:58
58      cmp      eax, ecx
+stepi 4
62      add      eax, 8      ; avanza 8 floats = 32 bytes
+p $ymm0.v8_float
$3 = {10, 12, 14, 16, 18, 20, 22, 24}
+p $rax
$4 = 8
```

En esta parte del GDB, se muestra primero el estado inicial de los registros rdi = dirección del arreglo, rsi = n = 16, rax = i = 0 y rcx = n & 7 = 16. En la primera iteración tras 4 instrucciones (cmp, jge, vmovups, vaddps), x/i \$pc muestra que la siguiente es add, se dice que vaddps ya se ejecutó. ymm1 tiene los 8 floats cargados y ymm0 = 0 + ymm1. En la segunda iteración ymm0 = 10, 12, 14, 16, 18, 20, 22, 24, se ve que cada carril sumó su propio elemento. Se hicieron 8 sumas independientes usando una sola instrucción.

```
+delete 1
+continue
Breakpoint 2, compute_stats.cs_p1_reduce () at asm/vector/stats_vector.asm:151
151      vextractf128 xmm3, ymm0, 1
+p $ymm0.v8_float
$5 = {10, 12, 14, 16, 18, 20, 22, 24}
+p $ymm1.v8_float
$6 = {1, 1, 1, 1, 1, 1, 1, 1}
+p $ymm2.v8_float
$7 = {9, 10, 11, 12, 13, 14, 15, 16}
+stepi 4
157      vextractf128 xmm3, ymm1, 1      ; xmm3 = carriles 4-7
+p $xmm0.v4_float
$8 = {136, 136, 136, 136}
```

```
+stepi 6
165      vextractf128 xmm3, ymm2, 1
+p $xmm1.v4_float
$9 = {1, 1, 1, 1}
+stepi 6
compute_stats.cs_p1_tail () at asm/vector/stats_vector.asm:175
175      cmp      eax, esi
+p $xmm2.v4_float
$10 = {16, 16, 15, 16}
```

En esta parte se hizo la reducción horizontal en compute-stats

```
+continue
Breakpoint 3, normalize_array.norm_done () at asm/vector/stats_vector.asm:342
342      vzeroupper
+p $xmm0.v4_float[0]
$11 = 8.5
+p $xmm1.v4_float[0]
$12 = 4.60977221
+p/x $rsi
$13 = 0x555555559480
+x/8fw $rsi
0x555555559480: -1.6269784      -1.41004801      -1.1931175      -0.97618705
0x555555559490: -0.759256601     -0.542326152     -0.325395674     -0.108465232
+x/16fw $rsi
0x555555559480: -1.6269784      -1.41004801      -1.1931175      -0.97618705
0x555555559490: -0.759256601     -0.542326152     -0.325395674     -0.108465232
0x5555555594a0: 0.108465232      0.325395674      0.542326152      0.759256601
0x5555555594b0: 0.97618705      1.1931175        1.41004801      1.6269784
+continue
[Inferior 1 (process 9828) exited normally]
```

En este bloque se revisó la salida después de la normalización

VII-B. Escalar

```
Breakpoint 1, sum_array.sum_loop () at asm/scalar/stats_scalar.asm:41
41      cmp      eax, esi      ; i < n ? (comparacion con signo -> jge)
+info registers rdi rsi rax
rdi      0x55555555a4c0      93824992257216
rsi      0x10                16
rax      0x0                0
+stepi 4
45      inc      eax          ; i++
+x/i $pc
=> 0x55555555d22 <sum_array.sum_loop+13>:      inc      %eax
+p $xmm1.v4_float
$1 = {1, 0, 0, 0}
+p $xmm0.v4_float
$2 = {1, 0, 0, 0}
+continue
Breakpoint 1, sum_array.sum_loop () at asm/scalar/stats_scalar.asm:41
+stepi 4
45      inc      eax          ; i++
+p $xmm0.v4_float
$3 = {3, 0, 0, 0}
+p $rax
$4 = 1
```

```

+delete 1
+continue
Breakpoint 2, compute_stats.cs_mean () at asm/scalar/stats_scalar.asm:100
100      cvtsi2ss xmm4, esi          ; xmm4 = (float) n  (entero -> float)
+p $rax
$5 = 16
+p $xmm0.v4_float
$6 = {136, 0, 0, 0}
+p $xmm1.v4_float
$7 = {1, 0, 0, 0}
+p $xmm2.v4_float
$8 = {16, 0, 0, 0}
+continue
Breakpoint 3, normalize_array.norm_done () at asm/scalar/stats_scalar.asm:197
197      ret
+p $xmm0.v4_float[0]
$9 = 8.5
+p $xmm1.v4_float[0]
$10 = 4.60977221
+p/x $rsi
$11 = 0x555555559480
+x/8fw $rsi
0x555555559480: -1.6269784      -1.41004801      -1.1931175      -0.97618705
0x555555559490: -0.759256601     -0.542326152     -0.325395674     -0.108465232

```

Se aprecia como se maneja un solo elemento por iteración y que en cada caso, en el registro xmm0 solo cambia el primer carril y rax avanza de 1 en 1. Aquí no existe reducción como ocurrió en el escalar, pues, todo ocurrió en un solo carril y los valores de salida coinciden con los de la versión vectorial.

Además, se hizo ejecución de GDB con un archivo con $N=21$, esto para observar la operación del remanente. Se observó que para $N = 16$, la versión escalar corrió 16 vueltas, mientras que el vectorial solo hizo 2, esto debido a que se hacen 8 operaciones por instrucción. Con $N = 21$, el escalar corrió 21 vueltas, mientras que el vectorial corrió 2 vueltas más 5 operaciones de remanente.

VIII. CONCLUSIONES, LIMITACIONES Y TRABAJO FUTURO

VIII-A. Conclusiones

En base a los resultados, se observó que usar vectorización fue entre 3.1 y 7.1 veces más rápida, esto por usar menos instrucciones y se estuvo muy cerca del 8x teórico (7.74x). En general, el proyecto cumple con tolerancias menores a 10^{-4} en los casos exigidos. También se observó que el speedup depende de donde se encuentren los datos dentro de la jerarquía de memoria, donde si está en L1 el limitante es el poder de cómputo, mientras que, si el dato está en RAM, la limitante es el ancho de banda de la memoria. Por último, también se puede mencionar que la versión vectorial es más precisa, repartir la suma en 8 acumuladores previene perder decimales en números grandes.

VIII-B. limitaciones

- Se observó que con N grandes se pierde mucha precisión, esto se debe a que la suma crece a valores donde float no puede representar el número real, el redondeo de cifras muy grandes es la causa aquí.
- Se tuvo desbordamiento con valores extremos, se llegó al límite de float 32 y los valores se hacían cero o no tenían sentido.
- El uso de un solo hilo limita la velocidad máxima con la que se podría hacer la operación, un solo núcleo podría no tener suficientes accesos para usar todo el ancho de banda de la RAM.

VIII-C. Trabajo futuro

Al código actual se le podrían hacer muchas mejoras si no se restringe al uso de números float32, se podrían haber utilizado los registros YMM, para hacer más sumas en paralelo y aumentar el rendimiento. El procesador usado tiene soporte para AVX-512, que presenta registros de 16 floats, lo que beneficiaría a N pequeños, para beneficiar también a tamaños de N grandes, se tendrían que usar más núcleos para aprovechar así más ancho de banda de la memoria. También, se podría modificar el código, para que en casos donde la varianza da inf o NaN, se reporte un error y no devuelva un arreglo de ceros.

REFERENCIAS

- [1] x86 psABIs, “System V Application Binary Interface: AMD64 Architecture Processor Supplement,” <https://gitlab.com/x86-psABIs/x86-64-ABI>, consultado el 1 de octubre de 2026.
- [2] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 2002.